HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA AN TOÀN THÔNG TIN



BÁO CÁO HỌC PHẦN

Đánh giá an toàn và bảo vệ dữ liệu người dùng ứng dụng OWASP Juice Shop v20.2.0

Học phần: DevOps và DevSecOps (SEC4349)

Thành viên	MSSV	Lỗi phụ trách
Nguyễn Toàn Thắng	B26CHAT039	F01 — SQL injection đăng nhập
Phạm Thị Minh Thúy	B26CHAT042	F02 — Poison Null Byte
Đỗ Đức Quốc Anh	B26CHAT001	F03 — lộ endpoint /metrics
Hồ Anh Vũ	B26CHAT048	F04 — cấu hình an toàn container

Năm học 2026

Mục lục

Danh mục từ viết tắt

1. Giới thiệu và phạm vi

2. Ứng dụng Juice Shop và dữ liệu cần bảo vệ

2.1. Nghiệp vụ đăng ký và đăng nhập

2.2. Nghiệp vụ tải tệp tài liệu

3. Môi trường triển khai và yêu cầu an toàn tối thiểu

3.1. Công cụ sử dụng

3.2. Chính sách và tiêu chuẩn đối chiếu

4. Quy trình DevSecOps và các cổng kiểm soát

4.1. Quét và phát hiện lỗi — kết quả thực tế

4.2. Bảng kết quả hợp nhất

4.3. Giám sát khi vận hành (pha Operate)

4.4. Tự động hóa: pipeline gắn với Git

5. Bốn lỗi an toàn: phát hiện và xử lý

5.1. F01 — SQL injection tại đăng nhập

5.2. F02 — Poison Null Byte làm lộ tệp sao lưu

5.3. F03 — Lộ endpoint /metrics không xác thực

5.4. F04 — Sai cấu hình an toàn của container

6. Kết luận

7. Tài liệu tham khảo

Phụ lục A — Mã nguồn chính sách Dockerfile (docker_security.rego)

Phụ lục B — Mã nguồn chính sách docker-compose (compose.rego)

Phụ lục C — Giải thích tiêu chuẩn và điều khoản tham chiếu

Danh mục từ viết tắt

Bảng 1. Danh mục từ viết tắt

Viết tắt	Giải nghĩa
SQLi	SQL Injection — tiêm nhiễm câu lệnh SQL
JWT	JSON Web Token — token phiên
SAST	Phân tích tĩnh mã nguồn (Static Application Security Testing)
CI/CD	Tích hợp / triển khai liên tục
OPA	Open Policy Agent (ngôn ngữ Rego, công cụ Conftest)
GDPR	Quy định bảo vệ dữ liệu của EU (2016/679)
PCI-DSS	Chuẩn an toàn dữ liệu thẻ thanh toán
CIS	Center for Internet Security (CIS Docker Benchmark)
CWE / OWASP	Phân loại lỗ hổng phần mềm / tổ chức an toàn ứng dụng web
UID	Mã định danh người dùng hệ điều hành

1. Giới thiệu và phạm vi

Báo cáo đánh giá an toàn ứng dụng **OWASP Juice Shop v20.2.0** dưới góc độ bảo vệ dữ liệu người dùng qua **bốn tình huống**. Bốn thành viên thực hiện, mỗi người phụ trách một tình huống: hai lỗi ở mã nguồn (F01, F02), một rủi ro phơi nhiễm ở khâu vận hành (F03) và một fixture cấu hình sai chủ động dựng để kiểm tra chính sách (F04). Mỗi tình huống được trình bày theo mạch **trước khi sửa → sau khi sửa**, gồm cách phát hiện, cách xử lý, nguyên tắc/tiêu chuẩn đối chiếu, pha DevSecOps tương ứng và đoạn mã demo.

Mỗi kịch bản được chạy song song trên hai môi trường để đối chiếu: bản **trước khi sửa** là container chạy image gốc (127.0.0.1:3100), còn bản **sau khi sửa** là bản triển khai đã qua reverse proxy (127.0.0.1:3102). Bên trong container, cổng 3000 là cổng mặc định của Juice Shop; bản đã xử lý vẫn lắng nghe cổng này nhưng không phơi ra host, chỉ reverse proxy (cổng 3102) truy cập được qua mạng nội bộ. Cổng 3100 dành cho container đối chứng chạy image gốc để so sánh. Juice Shop là ứng dụng cố ý chứa lỗ hổng phục vụ huấn luyện; các phát hiện ở đây minh họa quy trình, không nhằm đánh giá dự án gốc.

2. Ứng dụng Juice Shop và dữ liệu cần bảo vệ

Juice Shop là một ứng dụng web thương mại điện tử mô phỏng (bán nước ép) do OWASP phát hành nhằm huấn luyện an toàn ứng dụng. Về kiến trúc, đây là một Single Page Application gồm các thành phần sau:

Bảng 2. Thành phần của ứng dụng Juice Shop

Thành phần	Công nghệ	Vai trò
Giao diện người dùng	Angular (SPA)	Hiển thị, gọi REST API
Máy chủ ứng dụng	Node.js + Express	Xử lý REST API và nghiệp vụ
Truy xuất dữ liệu	ORM Sequelize trên SQLite	Lưu Users, Baskets, Cards…
Xác thực phiên	JWT (RS256), thư viện jsonwebtoken	Cấp và kiểm tra token đăng nhập
Giám sát	prom-client, endpoint `/metrics`	Xuất chỉ số dạng Prometheus
Đóng gói	Docker image distroless, chạy UID 65532	Triển khai bằng container

Một yêu cầu đi qua ứng dụng theo trình tự: trình duyệt tải **Angular** (SPA) rồi gọi **REST API**; **Express** tiếp nhận, với tài nguyên cần quyền thì lớp trung gian kiểm **JWT** (cấp lúc đăng nhập) trước khi cho qua; nghiệp vụ đọc/ghi dữ liệu qua **Sequelize trên SQLite**. Một phiên mua hàng điển hình: đăng nhập (mục 2.1) → duyệt sản phẩm → thêm vào **giỏ hàng** (Basket) → **đặt đơn và thanh toán** (Card/Order) — mỗi yêu cầu đều kèm JWT để ràng buộc đúng chủ sở hữu; song song, ứng dụng còn phục vụ **tệp tài liệu tĩnh** qua `/ftp` (mục 2.2) và mở sẵn **endpoint giám sát** `/metrics`. Bốn lỗi được phân tích rơi đúng các điểm trên luồng này: **F01** ở bước xác thực đăng nhập (REST API/ORM), **F02** ở route phục vụ tệp tĩnh, **F03** ở endpoint giám sát, **F04** ở cách đóng gói container. Yêu cầu bảo vệ dữ liệu được xác định theo GDPR (nguyên tắc ở Điều 5, nghĩa vụ an toàn ở Điều 32) và PCI-DSS cho dữ liệu thẻ thanh toán (chi tiết ở mục 3.2).

Bảng 3. Dữ liệu cần bảo vệ và cơ sở chính sách

Dữ liệu	Yêu cầu bảo vệ	Cơ sở chính sách
Email, hồ sơ	Chỉ chủ tài khoản/quản trị truy cập	GDPR Điều 5(1)(f), 25(2)
Mật khẩu	Không để lộ; không bị vượt xác thực	GDPR Điều 32(1)(b)
Token phiên (JWT)	Chỉ cấp khi xác thực đúng	GDPR Điều 32(1)(b)
Giỏ hàng	Ràng buộc chủ sở hữu ở mọi truy vấn	GDPR Điều 5(1)(f), 25(2)
Thẻ thanh toán	Không lộ số thẻ; PAN nếu lưu phải ở dạng không đọc được	PCI-DSS 3.4; GDPR Điều 32
Tệp cấu hình, sao lưu phía máy chủ	Không để tải về qua endpoint phục vụ tệp tĩnh	GDPR Điều 32(1)(b) (liên quan F02)

Ý nghĩa cụ thể của từng điều khoản (GDPR, PCI-DSS… và các chuẩn khác) được tóm gọn trong **Phụ lục C**.

2.1. Nghiệp vụ đăng ký và đăng nhập

Người dùng đăng ký qua `POST /api/Users` và đăng nhập qua `POST /rest/user/login`. Route `routes/login.ts` truy vấn bảng Users, nếu khớp thì cấp JWT kèm mã giỏ hàng. Vấn đề nằm ở ranh giới tin cậy: email do client gửi lên đi thẳng vào câu lệnh SQL, và đó chính là gốc của lỗi **F01**.

2.2. Nghiệp vụ tải tệp tài liệu

Thư mục `ftp/` gắn trực tiếp với bước **đặt đơn và thanh toán**: khi khách thanh toán xong, `routes/order.ts` sinh hóa đơn PDF `order_<mã đơn>.pdf` vào `ftp/`, ghi email khách, mã đơn, ngày đặt, danh sách sản phẩm và tổng tiền; giao diện chuyển khách tới `GET /ftp/order_<mã đơn>.pdf` để xem hóa đơn, và trang lịch sử đơn hàng cũng mở hóa đơn qua đường này. Cùng route còn phục vụ tài liệu công khai như điều khoản sử dụng `legal.md`. Route `routes/fileServer.ts` xử lý mọi yêu cầu `GET /ftp/:file` và chỉ cho phép đuôi `.md`/`.pdf`, nhưng **không kiểm JWT**: hóa đơn là dữ liệu cá nhân của khách mà không được ràng buộc chủ sở hữu như giỏ hàng hay đơn hàng, chỉ dựa vào mã đơn khó đoán, trong khi `/ftp` vẫn liệt kê toàn bộ thư mục. Thư mục `ftp/` còn chứa một số tệp **không nên để tải về** (bản sao lưu cấu hình, mã giảm giá…). Nếu bộ lọc đuôi file bị qua mặt, người dùng có thể đọc các tệp nhạy cảm đó — và đây là gốc của lỗi **F02**. Bản vá F02 xử lý lỗi vượt bộ lọc này; đưa hóa đơn ra sau kiểm tra JWT và chủ sở hữu đơn hàng là hướng mở rộng.

3. Môi trường triển khai và yêu cầu an toàn tối thiểu

Ứng dụng chạy bằng Docker trên Windows (WSL2) và được triển khai qua `docker-compose.yml`. Các yêu cầu an toàn tối thiểu ở đây bám theo CIS Docker Benchmark cùng nguyên tắc giảm bề mặt tấn công.

Để mỗi kết luận truy được về dữ liệu, thực nghiệm cố định các biến sau và chỉ thay đúng phần cần kiểm chứng (F01/F02 giữ nguyên seed/harness, chỉ thay route; F03 giữ logic ứng dụng, chỉ thay cấu hình truy cập; F04 chạy fixture trước/sau độc lập):

Bảng 4. Phương pháp thực nghiệm

Hạng mục	Giá trị cố định
Phiên bản mã trước khi vá	Bản mã Juice Shop gốc (thư mục `routes/`, `lib/` chưa chỉnh) — dùng để quét Semgrep và chạy ca kiểm thử đỏ
Phiên bản mã sau khi vá	Bản vá của nhóm, đóng gói trong image hotfix triển khai ở cổng 3102
Hai môi trường	Đối chứng `127.0.0.1:3100` (image gốc) ↔ triển khai `127.0.0.1:3102` (proxy + backend mạng internal)
Dữ liệu thử nghiệm	Bộ dữ liệu mẫu Juice Shop tự nạp khi khởi động (người dùng, giỏ hàng, sản phẩm giả lập)
Công cụ (phiên bản)	Semgrep 1.180.0; Gitleaks v8.30.1; Conftest/OPA v0.71.1; node:test + Supertest; Trivy 0.76.1; Dockle 0.4.15; Prometheus 3.15.1; Grafana 13.3.0
Tiêu chí đạt	F01/F02: ca kiểm thử xanh; F03: `/metrics` trả 404 từ ngoài; F04: conftest `exit 0` với bản tốt, `exit 1` với fixture lỗi
Bằng chứng	`report/evidence/*.log` (kèm exit code), kiểm thử, demo (tổng hợp ở mục 4.2)

Bảng 5. Kiểm soát an toàn tối thiểu

Kiểm soát	Cách thực hiện	Tiêu chuẩn
Giảm bề mặt mạng	Chỉ reverse proxy publish cổng của **stack ứng dụng**, trên 127.0.0.1; backend ở mạng internal	GDPR Điều 32
Không chạy root	Juice Shop UID 65532, Nginx UID 101	CIS Docker 4.1
Đặc quyền tối thiểu	`cap_drop: ALL`, `no-new-privileges`; hệ thống tệp gốc đặt chế độ chỉ đọc ở các service hỗ trợ	CIS Docker 5.3/5.12
Ghim phiên bản	Luật Rego chặn `:latest`, cảnh báo khi base image chưa ghim digest	CIS Docker Benchmark §4
Lọc đường vào	Nginx chặn /metrics, giới hạn 5 lần đăng nhập/phút	GDPR Điều 32

Những kiểm soát này được **xác minh ngay trên container đang chạy** bằng `docker inspect` (Compliance as Code), cho thấy chúng thực sự có hiệu lực chứ không chỉ nằm trên file cấu hình:



Hình 1. Bằng chứng runtime qua `docker inspect`: cả ba container chạy non-root (USER), `cap_drop: ALL`, `no-new-privileges`; backend không mở cổng host (CỔNG HOST = "-", mạng internal), chỉ proxy mở loopback 127.0.0.1:3102

3.1. Công cụ sử dụng

Bảng 6. Công cụ và vai trò

Công cụ	Loại	Vai trò trong báo cáo
Docker, Docker Compose	Nền tảng container	Triển khai ứng dụng và môi trường kiểm thử
Nginx	Reverse proxy	Chặn /metrics, giới hạn tốc độ đăng nhập (F03)
Semgrep	SAST (phân tích tĩnh)	Đọc mã nguồn (không chạy) để chỉ ra các mẫu lỗi; trong phạm vi báo cáo, xác định chính xác dòng SQL injection của F01 (đăng nhập) và Poison Null Byte của F02 (route tải tệp)
Gitleaks	Quét secret	Tìm khóa/bí mật lộ trong mã nguồn
node:test (Supertest)	Kiểm thử động	Chạy ứng dụng thật, tái hiện lỗi *trước khi vá* (đỏ) và kiểm chứng *sau khi vá* (xanh)
Conftest / OPA (Rego)	Policy as Code	Kiểm tra *văn bản* Dockerfile và cấu hình triển khai (pha Build/Deploy, F03/F04)
Trivy	Quét image (SCA)	Quét CVE dependency + secret + misconfig trên **image đã build** (pha Release); bổ sung mảng SCA mà các công cụ trên không có
Dockle	Lint image (CIS)	Kiểm cấu hình **image sau build** theo CIS Docker — bắt secret bake vào layer (tệp & `ENV`) và user chạy thật/root (pha Release)
Prometheus + Grafana	Giám sát (Operate)	Thu thập `/metrics` nội bộ, cảnh báo bất thường, dashboard vận hành
Git hook + GitHub Actions	Tự động hóa (CI)	Gọi `devsecops/pipeline.sh` để chạy toàn bộ cổng trên theo thứ tự 7 pha mỗi khi commit, push và trên GitHub (mục 4.4)

3.2. Chính sách và tiêu chuẩn đối chiếu

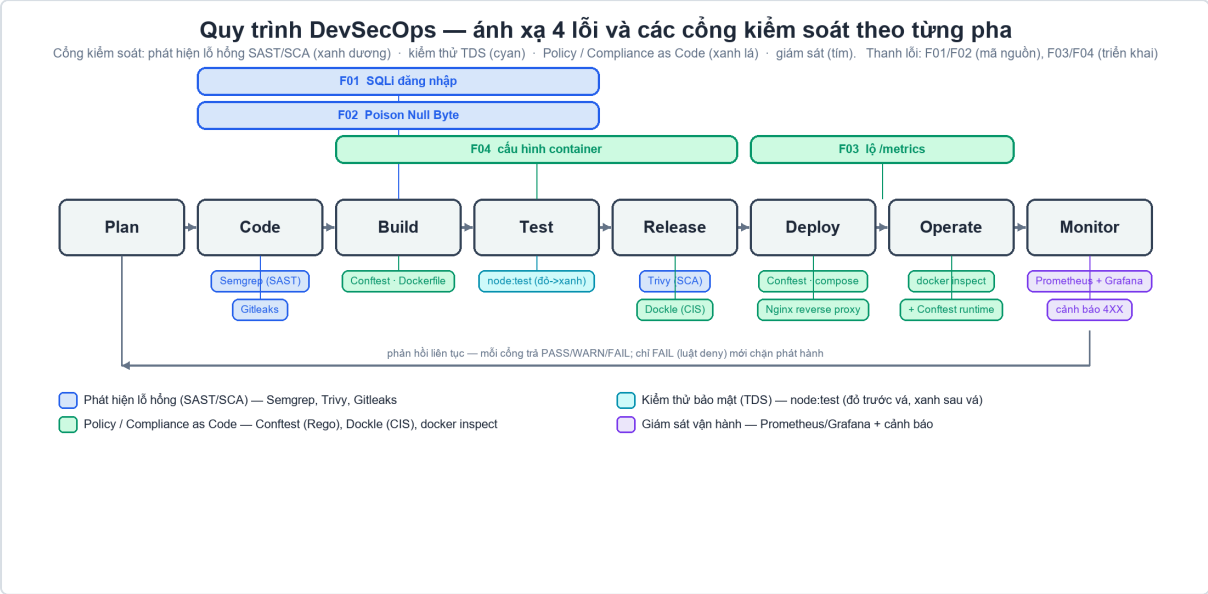
Báo cáo đối chiếu các **tiêu chuẩn quốc tế** về bảo vệ dữ liệu cá nhân; nhóm dựng một **Dockerfile có lỗi** (`report/fixtures/Dockerfile.vulnerable`) cùng bộ luật Rego để kiểm chính sách trong môi trường lab. Phần thân chỉ ghi **mã điều khoản — ý nghĩa (yêu cầu gì) của mọi điều khoản tra tại Phụ lục C**.

- GDPR (EU) 2016/679 — Điều 5, 25, 32; PCI-DSS — Yêu cầu 2.2, 3.4, 8.2.1; OWASP Top 10 2021 & CWE — A03/CWE-89 (F01), A01/CWE-158 + CWE-180 (F02); CIS Docker Benchmark — 4.1, 4.9, 4.10, 5.3, 5.4, 5.8, 5.12 (F04); OCI image-spec, NIST SP 800-190, NIST SSDF (SP 800-218), SLSA (chuỗi cung ứng — chỉ *tham chiếu*, không tuyên bố đạt mức).

Với dữ liệu giả lập của ứng dụng huấn luyện, báo cáo dùng cách diễn đạt *"đối chiếu nguyên tắc bảo vệ"* thay cho kết luận tuân thủ pháp luật.

4. Quy trình DevSecOps và các cổng kiểm soát

Việc phát hiện và chống tái diễn các lỗi được thực hiện theo **Security as Code** — cách làm **viết mọi kiểm soát an toàn thành mã** (luật, kịch bản quét, ca kiểm thử) đặt trong kho mã và chạy **tự động như các cổng (gate)** trong pipeline CI/CD. Vì là mã nên các kiểm soát này **xem lại, lặp lại và truy vết được**, và có thể **chặn phát hành ngay khi phát hiện vi phạm** (shift-left) thay vì chỉ rà soát thủ công một lần. Hình dưới đặt các cổng kiểm soát lên vòng đời DevSecOps:



Hình 2. Quy trình DevSecOps: ánh xạ bốn lỗi F01–F04 và các cổng kiểm soát theo pha — phát hiện lỗ hổng SAST/SCA (Semgrep, Trivy), kiểm thử bảo mật TDS (node:test), Policy/Compliance as Code (Conftest, Dockle, docker inspect) và giám sát vận hành (Prometheus/Grafana)

Các nhóm kỹ thuật trong khung, xếp theo **pha vòng đời**:

- **Phát hiện lỗ hổng (SAST/SCA):** Semgrep quét tĩnh mã nguồn, Trivy quét CVE *dependency* của image, Gitleaks quét secret trong commit.
- **Kiểm thử bảo mật (TDS):** ca kiểm thử bảo mật tự động — đỏ khi còn lỗi, xanh sau khi vá — giữ lại để chống tái diễn.
- **Policy as Code (Build/Deploy):** luật Rego kiểm *văn bản* cấu hình (Dockerfile, `docker-compose`), chặn cấu hình không an toàn **trước khi triển khai**.
- **Compliance as Code (Operate):** kiểm **container đang chạy** bằng `docker inspect` + luật Rego runtime, và **Dockle** kiểm CIS trên **image thành phẩm**.

- **Giám sát vận hành (Operate): Prometheus** scrape `/metrics` nội bộ, **Grafana** dashboard, và **cảnh báo** khi có bất thường — mục 4.3.

Toàn bộ kiểm soát được viết thành mã nên chạy lặp lại được trên **hai môi trường song song**: container đối chứng *trước vá* (`127.0.0.1:3100`) và bản triển khai *sau vá* qua reverse proxy (`127.0.0.1:3102`, backend ở mạng internal).

Quy trình tổ chức theo **bảy pha** vòng đời, mở rộng cho phù hợp phạm vi dữ liệu cá nhân: thêm quét **SCA (Trivy)** ở pha 3, kiểm **image theo CIS (Dockle)** ở pha 6, và **giám sát vận hành** ở pha 7. Hình trên ánh xạ các cổng này lên vòng đời DevSecOps.

Bảng 7. Bảy pha thực hành ánh xạ vào bốn tình huống

Pha	Hoạt động	Công cụ nhóm dùng	Tình huống
1. Thiết lập môi trường	Dựng hai môi trường trước/sau vá (3100 đối chứng, 3102 proxy)	Docker Compose, Node.js	—
2. Policy as Code	Kiểm chính sách Dockerfile + compose **trước** khi triển khai	Conftest/OPA (Rego)	F03, F04
3. Phát hiện lỗ hổng & ánh xạ điều khoản	Quét tĩnh mã nguồn (SAST) + quét CVE dependency (SCA), gắn với GDPR/PCI-DSS/CWE	Semgrep, Trivy	F01, F02
4. Test-Driven Security	Kiểm thử đỏ trước vá, xanh sau vá	node:test + Supertest	F01, F02
5. Vá ứng dụng	Sửa mã theo kết quả kiểm thử	TypeScript / Express	F01, F02
6. Compliance as Code	Kiểm image thành phẩm (CIS) và **container đang chạy** bằng `docker inspect` + Conftest runtime + Dockle	Dockle, `docker inspect`, Conftest	F03, F04
7. Kiểm chứng & bàn giao	Chạy lại toàn bộ cổng + quét secret; **giám sát vận hành**	Gitleaks, Conftest; Prometheus/Grafana	—

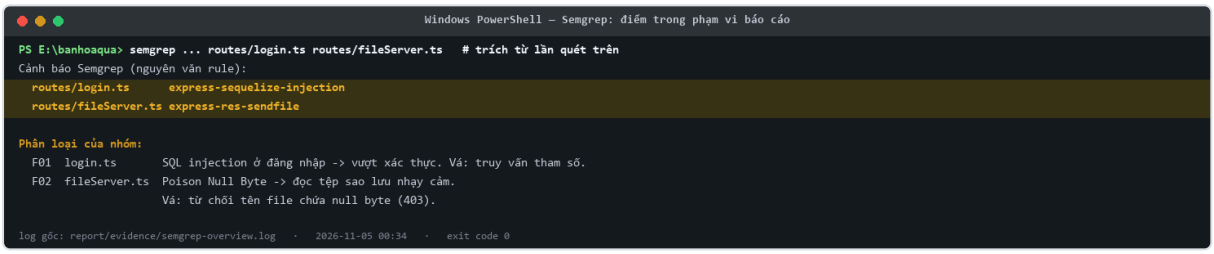
Hai cổng kiểm soát bổ trợ đều cho **exit 0** (log gốc kèm thời điểm, phiên bản công cụ, mã thoát): **Gitleaks** quét các commit vá của nhóm — không phát hiện secret (`gitleaks.log`); **Conftest verify** — 19/19 unit test của hai bộ luật Dockerfile và compose đạt, gồm 13 ca Dockerfile và 6 ca compose (`conftest-verify.log`); tính cả 7 ca của luật runtime (`report/policy/runtime_test.rego`) là **26/26**, đúng con số pipeline chạy ở pha Build (mục 4.4).

Mỗi lỗi đi kèm một script demo (`demos/tv1`…`tv4`) tái hiện lỗi trước khi vá và kiểm chứng sau khi vá; các script này được trình bày trong từng mục tương ứng ở phần 5.

4.1. Quét và phát hiện lỗi — kết quả thực tế

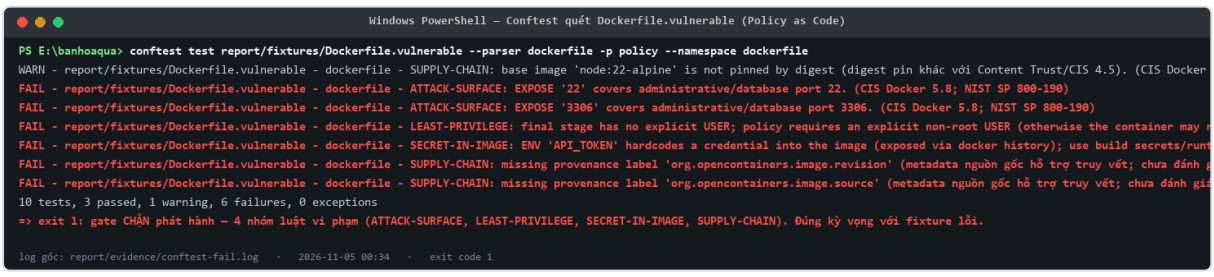
Các cổng quét tự động được chạy trên mã nguồn và cấu hình *chưa vá* để lộ ra lỗ hổng trước khi sửa.

SAST (Semgrep). Quét toàn bộ `routes/` và `lib/` (mã *chưa vá*) cho ra **38 cảnh báo** trên 25 tệp (SQL/NoSQL injection, XSS, băm yếu, JWT secret cố định…). Đây là **cảnh báo công cụ, chưa phải 38 lỗ hổng đã xác nhận**; mỗi cảnh báo là một điểm cần rà soát thêm. Thuộc phạm vi báo cáo, phân tích tĩnh chỉ ra hai điểm: **F01** tại `login.ts` (rule `express-sequelize-injection` — SQL injection ở đăng nhập) và **F02** tại `fileServer.ts` (rule `express-res-sendfile` — Poison Null Byte, lộ tệp sao lưu).



Hình 3. Hai điểm Semgrep xác định trong phạm vi: F01 SQL injection ở đăng nhập (`login.ts`) và F02 Poison Null Byte (`fileServer.ts`); kèm phân loại của nhóm

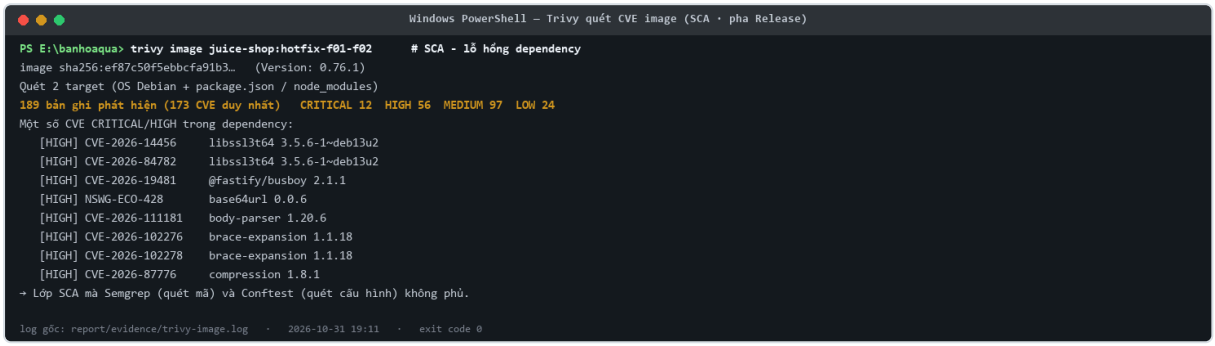
Policy as Code (Conftest). Bộ luật Rego được **định nghĩa thành mã và kiểm thử trước** (cách xây dựng ở mục 5.4), sau đó Conftest **đối chiếu** cấu hình với bộ luật đó. Đối chiếu `Dockerfile.vulnerable` với bộ luật của nhóm (Phụ lục A) cho `exit 1` với **6 thông báo vi phạm thuộc 4 nhóm luật** `deny`; mã thoát đủ điều kiện làm cổng chặn:



Hình 4. Conftest quét Dockerfile.vulnerable: `exit 1`, 6 thông báo vi phạm thuộc 4 nhóm luật `deny`; mã thoát đủ để làm cổng chặn

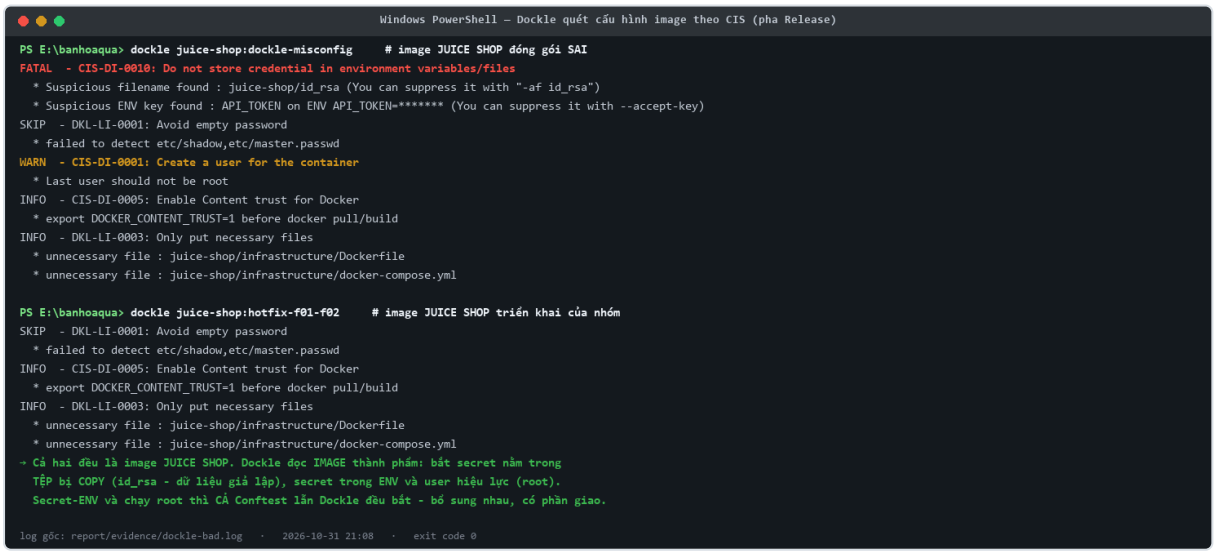
Quét image — CVE/SCA (Trivy, pha Release). Trên image triển khai `juice-shop:hotfix-f01-f02`, Trivy quét lỗ hổng *dependency*: **189 bản ghi phát hiện** (173 mã CVE duy nhất; 12 CRITICAL, 56 HIGH, 97 MEDIUM, 24 LOW) — gồm các thư viện cũ như `crypto-js` (CVE-2023-46233, CRITICAL), `jws`, `body-parser`, `brace-expansion`.

Đây là **mảng SCA** mà Semgrep (quét mã nhóm tự viết) và Conftest (quét cấu hình) **không phủ**. Nhóm ghi nhận là **rủi ro đã biết**, xử lý bằng nâng cấp dependency khi triển khai thật — vá toàn bộ nằm ngoài phạm vi bốn tình huống (JSON gốc: `report/evidence/trivy-image.json`) (Hình 5).



Hình 5. Trivy quét lỗ hổng dependency của image hotfix: 189 bản ghi (173 CVE duy nhất; 12 CRITICAL, 56 HIGH); SCA là lớp mà SAST/Policy-as-Code không phủ

Quét image — CIS (Dockle, pha Release). Dockle đọc *image đã build* (không phải văn bản Dockerfile) nên bắt được lỗi chỉ lộ ở thành phẩm. Để kiểm chứng ngay **trên image Juice Shop**, nhóm lấy chính image triển khai rồi chồng thêm cấu hình sai (chạy root, COPY `id_rsa`, secret trong `ENV`) thành một image Juice Shop đóng gói sai. Trên image này, Dockle báo **FATAL** (`CIS-DI-0010`) cho **cả secret nằm trong tệp bị COPY vào layer** (`id_rsa` — dữ liệu giả lập) **lẫn secret trong `ENV`** (`API_TOKEN`), kèm **WARN** chạy root (`CIS-DI-0001`); còn image triển khai của nhóm **không có FATAL/WARN** (chỉ còn INFO và một kiểm tra bị bỏ qua do không đọc được `/etc/shadow`) (Hình 6).



Hình 6. Dockle quét trên image Juice Shop: bản đóng gói sai bị FATAL (secret trong tệp id_rsa giả lập + secret ENV) và WARN chạy root; image triển khai của nhóm không có FATAL/WARN

Tái lập: các lệnh quét (Semgrep, Gitleaks, Conftest, Trivy, Dockle, `docker inspect`, Prometheus, Grafana) được gom ở tài liệu ngoài `report/HUONG_DAN_DEMO.md` và `report/scanSAC.md`; log gốc kèm mã thoát ở `report/evidence/`, ảnh dựng từ `report/figures/capture_runs.py` và `capture_scans.py`.

4.2. Bảng kết quả hợp nhất

Bảng dưới liên kết từng tình huống với kết quả và **tệp bằng chứng** tương ứng (log gốc trong `report/evidence/`, kiểm thử, demo).

Bảng 8. Kết quả hợp nhất bốn tình huống (điều kiện → kỳ vọng → trước/sau vá → bằng chứng)

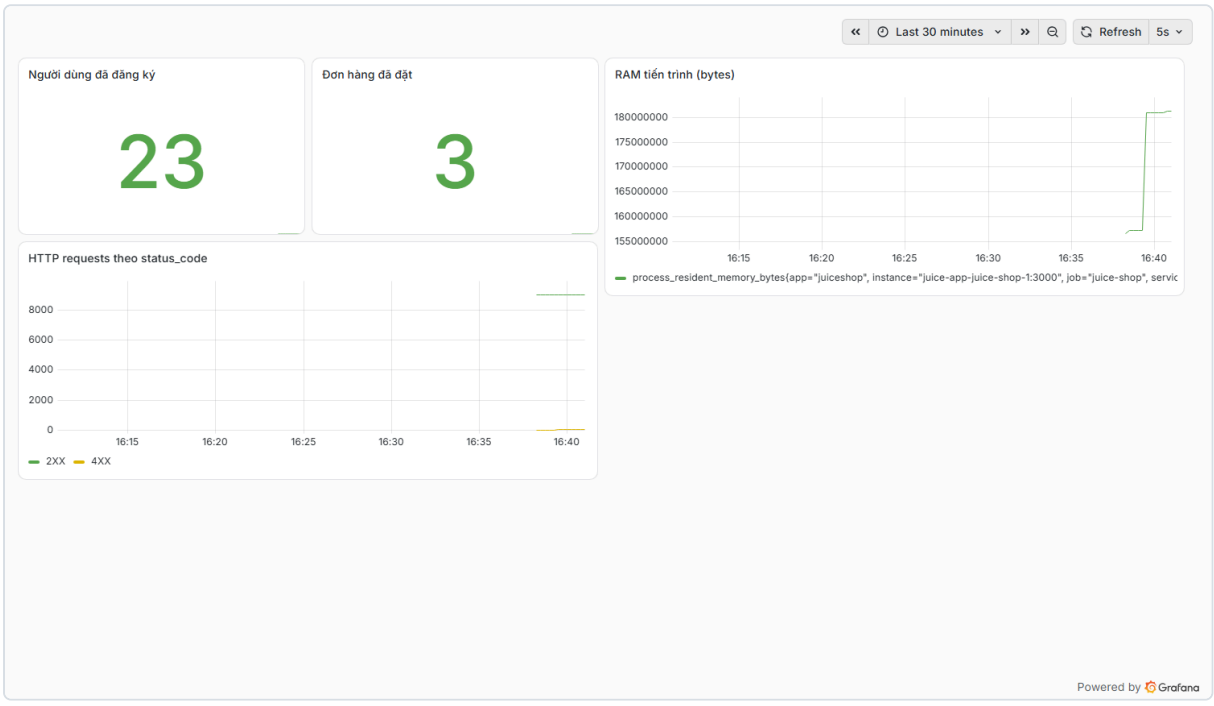
Mã	Điều kiện kiểm thử	Kết quả mong đợi	Trước vá	Sau vá	Bằng chứng
F01	Đăng nhập bằng payload SQL + mật khẩu sai	Từ chối (401)	Cấp token, vượt xác thực	401; đăng nhập hợp lệ vẫn 200	`f01-f02.test.ts`; demo `tv1`; `semgrep-overview.log`
F02	`GET /ftp/<tên>%2500.md` (poison null byte)	Từ chối tệp đuôi bị cấm	Đọc được `package.json.bak`, `coupons_2013.md.bak`	403; tệp `.md` hợp lệ vẫn tải	`f01-f02.test.ts`; demo `tv2`; `tv2-f02.log`
F03	`GET /metrics` từ host	Chặn truy cập từ ngoài	200, lộ chỉ số vận hành	404 qua proxy; backend không mở cổng host	demo `tv3`; `env-*.log`; `hardening`
F04	Conftest trên fixture lỗi / bản vá / Dockerfile gốc	Chặn fixture lỗi, đạt bản tốt	Fixture: `exit 1`, 6 vi phạm / 4 nhóm	Bản vá & gốc: đạt; `verify` 19/19	`conftest-fail.log`; `conftest-verify.log`; demo `tv4`

Các chỉ số ở F03 là số liệu tổng hợp, chưa phải dữ liệu cá nhân định danh; F04 là fixture cố ý lỗi, không phải lỗi image gốc.

4.3. Giám sát khi vận hành (pha Operate)

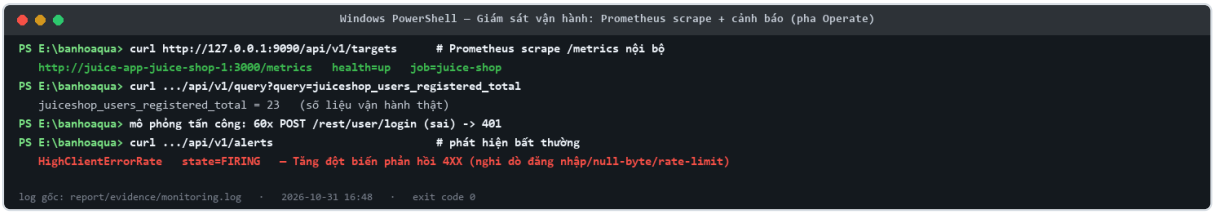
Các lớp trên dừng ở "trước khi chạy" và "cấu hình lúc chạy". Khi ứng dụng **đang chạy**, nhóm bổ sung một lớp **quan sát + phát hiện** nhẹ: **Prometheus** scrape endpoint `/metrics` của ứng dụng qua **mạng nội bộ** (app vẫn không mở cổng host — đúng nguyên tắc F03), **Grafana** làm portal hiển thị, và một **luật cảnh báo** để phát hiện bất thường.

Chính `/metrics` mà F03 chặn ra ngoài lại là *nguồn cấp dữ liệu giám sát nội bộ*: nó xuất số người dùng, số đơn hàng, thời gian xử lý HTTP và số phản hồi theo mã trạng thái. Prometheus scrape endpoint này (target **UP**), Grafana vẽ thành dashboard vận hành (Hình 7).



Hình 7. Dashboard Grafana (portal giám sát) với số liệu thật từ ứng dụng đang chạy: người dùng đã đăng ký, đơn hàng, HTTP theo status_code, RAM tiến trình

Phần **phát hiện**: luật cảnh báo `HighClientErrorRate` theo dõi **số phản hồi 4XX tại backend** (ngưỡng `>20` trong 2 phút). Khi nhóm gửi 60 yêu cầu đăng nhập sai **trực tiếp tới backend** (mô phỏng dò mật khẩu), số 4XX vọt lên và cảnh báo **chuyển trạng thái FIRING** (Hình 8) — minh họa khả năng phát hiện bất thường ở tầng quan sát mà mã nguồn không tự biết. Đây là ngưỡng tổng quát trên backend (chưa phân biệt endpoint hay nguồn truy cập); FIRING là *trạng thái của luật*, còn việc giám sát các yêu cầu bị Nginx chặn ngay tại proxy (vd rate-limit 429) và định tuyến thông báo tới người vận hành (Alertmanager) thuộc hướng mở rộng.



Hình 8. Giám sát vận hành: Prometheus scrape /metrics (target UP), truy vấn số liệu thật, và cảnh báo HighClientErrorRate chuyển FIRING khi mô phỏng dò đăng nhập

Lưu ý ranh giới: stack giám sát mở thêm hai cổng loopback cho người vận hành — `127.0.0.1:9090` (Prometheus) và `127.0.0.1:3103` (Grafana, bật anonymous access phục vụ demo lab) — và chạy ở compose riêng chưa thuộc phạm vi `compose.rego`. Vì vậy phát biểu "chỉ reverse proxy publish cổng" ở mục 3 áp dụng cho **stack ứng dụng**; chính stack giám sát hiện còn vi phạm luật `F03 NET-02` của nhóm (Prometheus, Grafana mở cổng mà không phải dịch vụ biên) — ghi nhận là giới hạn cần khắc phục.

4.4. Tự động hóa: pipeline gắn với Git

Các cổng ở mục 4.1–4.3 không chạy tay từng lệnh mà được gói vào **một script** `devsecops/pipeline.sh`, chạy theo đúng thứ tự 7 pha của Hình quy trình. Git gọi script này ở ba điểm (bảng dưới; hai hook trên máy có thể bị bỏ qua bằng `--no-verify`), nên máy thành viên và GitHub chạy **cùng một bộ cổng**; cổng nào FAIL thì script trả `exit 1` và Git hủy thao tác (hoặc step trên GitHub chuyển đỏ).

Bảng 9. Ba điểm kích hoạt pipeline

Kích hoạt	Chạy gì	Thời gian	Bỏ qua được?
`git commit` → hook `.githooks/pre-commit`	Cổng tĩnh trên phần đã `git add`: Gitleaks, Semgrep, Conftest Dockerfile + compose	vài giây – 1 phút	Được
`git push` → hook `.githooks/pre-push`	Đủ 7 pha trên các commit sắp đẩy: build image, kiểm thử, quét image, triển khai lên 3102 rồi kiểm tra vận hành	khoảng 2 phút	Được
GitHub Actions `.github/workflows/devsecops.yml`	Đủ 7 pha, mỗi pha một step, triển khai trên máy ảo của GitHub; lưu log từng cổng làm artifact	khoảng 5 phút	**Không**

Bảng 10. Cổng của từng pha trong pipeline

Pha	Cổng	Chặn khi
Code	Gitleaks · Semgrep	Có secret trong dòng **được thêm**; có cảnh báo SAST **mới**
Build	`conftest verify` (26 ca) · `docker_security.rego` · `tsc` · `docker build`	Luật Rego sai; Dockerfile vi phạm; lỗi biên dịch
Test	node:test + Supertest	Một ca kiểm thử bảo mật đỏ
Release	Trivy · Dockle → gắn tag phát hành	CVE CRITICAL **mới**; CIS WARN/FATAL
Deploy	`compose.rego` · `docker compose up --wait`	Compose vi phạm; container không healthy
Operate	`runtime.rego` trên `docker inspect` · gọi HTTP qua proxy	Container chạy root/thiếu `cap_drop`…; `/metrics` ≠ 404; thiếu header; không có 429
Monitor	Prometheus	Target không UP; chưa nạp luật cảnh báo

Ba quyết định thiết kế giúp pipeline dùng được trên một ứng dụng **cố ý chứa lỗ hổng**:

- **Chỉ chặn cái mới.** `devsecops/semgrep-diff.mjs` so cảnh báo Semgrep của bản mới với bản gốc của cùng tệp (nhận diện theo luật + nội dung dòng, không theo số dòng), nên 38 cảnh báo có sẵn không chặn mọi thay đổi về sau; Gitleaks chỉ quét dòng được thêm; Trivy bỏ qua 11 CVE CRITICAL có sẵn của image gốc, liệt kê kèm gói và bản sửa trong `devsecops/.trivyignore`.
- **Image chỉ được phát hành khi qua cổng.** Pha Build tạo image *ứng viên* `juice-shop:ci-<sha>`; chỉ khi Test và Release đều đạt mới gắn tag `juice-shop:hotfix-f01-f02` mà compose triển khai — bản lỗi không thể lên 3102.
- **Lọc theo loại thay đổi.** Chỉ sửa tài liệu thì chỉ chạy Gitleaks; sửa mã hoặc hạ tầng thì chạy đủ. Pha nào FAIL thì các pha sau dừng (riêng pre-commit chạy hết cổng tĩnh để thấy mọi lỗi một lần).

Mỗi tình huống dưới đây được thử bằng **`git commit`** / **`git push`** **thật** (kịch bản ở `report/scanSAC.md`), sau đó trả repo về trạng thái sạch:

Bảng 11. Kiểm chứng tích hợp — đưa lỗi trở lại và quan sát cổng chặn

Tình huống	Bị chặn ở	Kết quả
Đưa lại `login.ts` chưa vá (F01)	commit · Code	Semgrep: 1 cảnh báo mới `express-sequelize-injection`
Ghi khóa AWS giả vào mã	commit · Code	Gitleaks: leaks found 1
Đưa lại `fileServer.ts` chưa vá (F02)	push · Test	Commit qua (Semgrep không thấy); ca null byte nhận 200 thay vì 403
Backend mở cổng `0.0.0.0:3000` (F03)	commit · Deploy	`compose.rego`: `F03 NET-02` + `NET-01`
Proxy bỏ chặn `/metrics` (F03)	push · Operate	Không luật tĩnh nào đọc `nginx.conf`; sau triển khai `GET /metrics` → 200
Dockerfile bỏ `USER` (F04)	commit · Build	`docker_security.rego`: LEAST-PRIVILEGE
Container chạy root, bỏ `cap_drop` (F04)	push · Operate	`runtime.rego`: PRIV-02, PRIV-03
Đẩy bản đã vá	—	pre-push: 21 PASS · 0 FAIL, khoảng 1 phút 40 giây
GitHub Actions trên `main` (`full=true`)	—	Đủ 7 pha, mọi step xanh

Hook trên máy có thể bị bỏ qua bằng `--no-verify`, nên cổng bắt buộc cuối cùng là GitHub Actions; để workflow thực sự chặn được việc merge, nhánh `main` cần bật branch protection yêu cầu job *DevSecOps pipeline* đạt — việc này chưa làm.

5. Bốn lỗi an toàn: phát hiện và xử lý

5.1. F01 — SQL injection tại đăng nhập

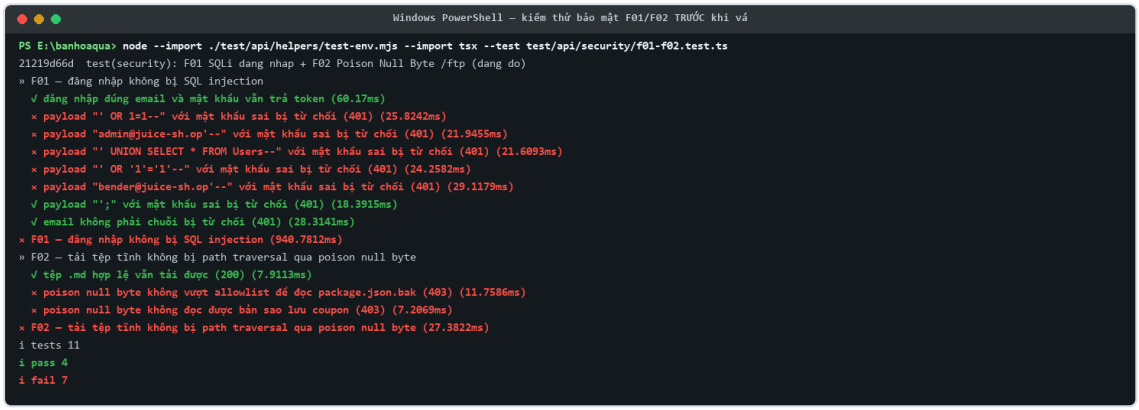
Trước khi sửa

Route `routes/login.ts` ghép thẳng email vào câu SQL:

```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}'
  AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model:
UserModel, plain: true })
```

Với email `' OR 1=1--`, mệnh đề WHERE luôn đúng còn phần kiểm tra mật khẩu bị đẩy thành chú thích, nên máy chủ cấp token của tài khoản đầu tiên (tài khoản quản trị) mà không cần mật khẩu. Đây chính là CWE-89, OWASP A03:2021.

Phát hiện. Semgrep (luật `express-sequelize-injection`) chỉ đúng dòng lỗi (xem hình quét tổng ở mục 4.1). Kiểm thử `test/api/security/f01-f02.test.ts` gửi payload kèm mật khẩu sai và yêu cầu phản hồi 401; trên mã chưa sửa, test báo đỏ (Hình 9). Demo `demos/tv1_f01_sqli.py` xác nhận token vẫn được cấp (Hình 10).



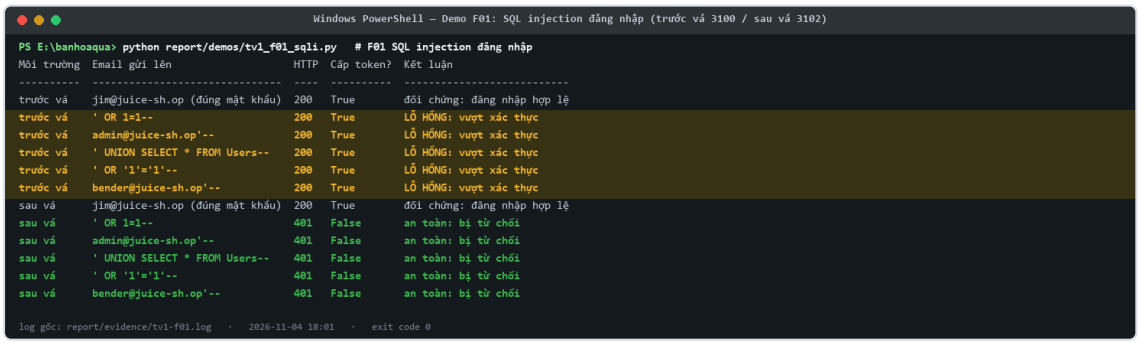
Hình 9. Kiểm thử bảo mật F01/F02 trên mã chưa sửa: các ca kiểm thử đỏ ứng với lỗ hổng còn tồn tại

Sau khi sửa

Dùng truy vấn tham số (bind parameter):

```
const email = typeof req.body.email === 'string' ? req.body.email : ''
const password = typeof req.body.password === 'string' ? req.body.password : ''
models.sequelize.query('SELECT * FROM Users WHERE email = $email AND password = $password AND
deletedAt IS NULL',
  { bind: { email, password: security.hash(password) }, model: UserModel, plain: true })
```

Câu lệnh được biên dịch trước và giá trị truyền riêng, nên mọi ký tự SQL trong email giờ chỉ còn được hiểu là dữ liệu. Sau khi sửa, kiểm thử bảo mật xanh toàn bộ (Hình 12); demo cũng cho thấy mọi payload đều bị từ chối 401 trong khi đăng nhập hợp lệ vẫn hoạt động bình thường (Hình 10).



Hình 10. Demo F01: nhiều biến thể payload SQLi — trước khi sửa đều được cấp token, sau khi sửa đều bị từ chối

Nguyên tắc/tiêu chuẩn đối chiếu: GDPR Điều 32(1)(b) (bảo mật khi xử lý dữ liệu). **Pha DevSecOps:** Code → Test. **Kiểm soát (Security as Code):** phát hiện bằng SAST ở pha

Code, chặn tái diễn bằng kiểm thử bảo mật tự động ở pha Test.

5.2. F02 — Poison Null Byte làm lộ tệp sao lưu

Trước khi sửa

Route `routes/fileServer.ts` phục vụ tệp trong thư mục `ftp/` (hóa đơn sinh ra khi thanh toán và tài liệu công khai, mục 2.2) qua `GET /ftp/:file`. Ứng dụng kiểm tra hậu tố `.md`/`.pdf` (ngoài ra có ngoại lệ cho `incident-support.kdbx`), sau đó mới phục vụ:

```
if (file && (endsWithAllowlistedFileType(file) || (file === 'incident-support.kdbx'))) {
  file = security.cutOffPoisonNullByte(file)   // cắt tên tại chuỗi literal "%00" rồi mới phục vụ
  res.sendFile(path.resolve('ftp/', file))
}
```

Điểm yếu nằm ở **thứ tự xử lý**: tên tệp được *kiểm tra* khác với tên *thực sự dùng để đọc*. Với yêu cầu `/ftp/package.json.bak%2500.md`, quá trình giải mã URL tạo tham số `package.json.bak%00.md`; tham số này vượt qua kiểm tra hậu tố `.md`, rồi hàm `cutOffPoisonNullByte` cắt tại chuỗi `%00` thành `package.json.bak` và chuyển cho `res.sendFile`. Nhờ đó người dùng **đọc được các tệp sao lưu phía máy chủ** mà bộ lọc đuôi lẽ ra chặn — ví dụ `package.json.bak` (thông tin cấu hình, phiên bản dependency) hay `coupons_2013.md.bak` (mã giảm giá). Đây là lỗi **Poison Null Byte** (CWE-158; CWE-180 kiểm tra trước khi chuẩn hóa; OWASP A01:2021 Broken Access Control).

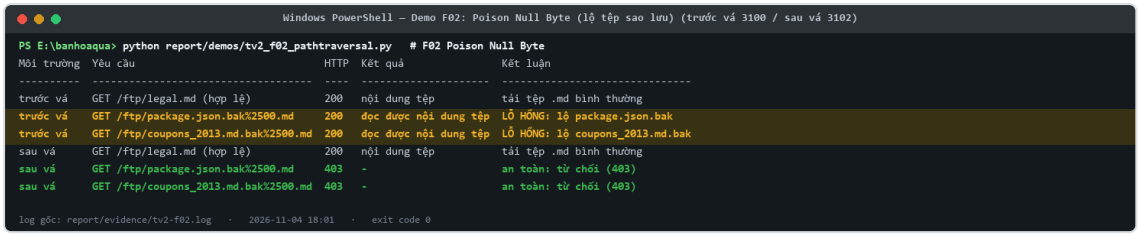
Phát hiện. Semgrep (luật `express-res-sendfile`) chỉ ra lời gọi `res.sendFile` nhận tên tệp từ đầu vào người dùng — F02 là một trong các cảnh báo SAST được xác nhận (xem hình quét ở mục 4.1). Vì cảnh báo chỉ nêu vị trí, nhóm phân tích luồng xử lý và dùng **kiểm thử HTTP** để xác nhận việc vượt bộ lọc: kiểm thử `test/api/security/f01-f02.test.ts` gửi `GET /ftp/package.json.bak%2500.md` và yêu cầu phản hồi 403; trên mã chưa sửa, ca này báo đỏ (Hình 9). Demo `demos/tv2_f02_pathtraversal.py` cho thấy cùng payload đọc được nội dung tệp ở bản chưa vá (Hình 11).

Sau khi sửa

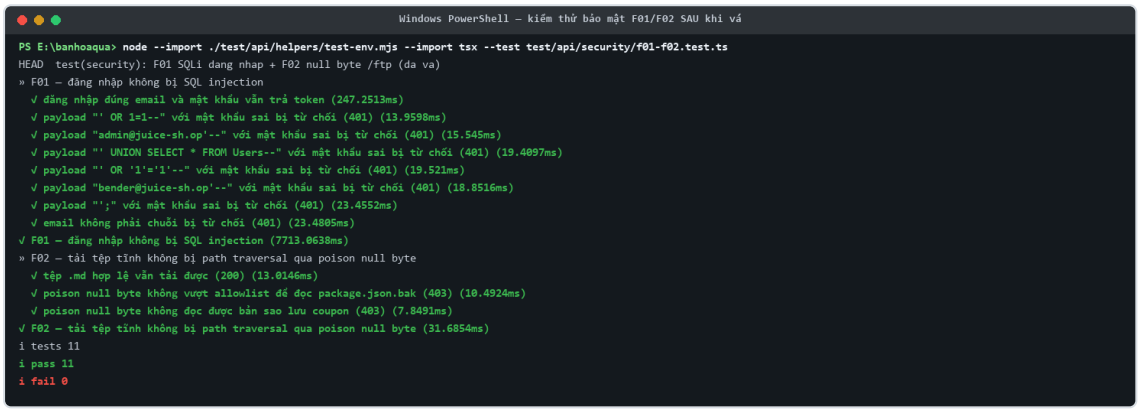
Từ chối hẳn tên tệp chứa null byte (`%00` dạng chuỗi hoặc ký tự NUL thực `\u0000`) **trước** bước kiểm tra loại và phục vụ, thay vì cắt bỏ rồi gửi:

```
if (file && (file.includes('%00') || file.includes('\u0000'))) {
  res.status(403)
  next(new Error('File names cannot contain null bytes!'))
  return
}
```

Sau khi sửa, hai payload `...%2500.md` bị từ chối **403**, trong khi tải tệp hợp lệ (`legal.md`) vẫn trả về bình thường (Hình 11); kiểm thử bảo mật khi đó xanh toàn bộ (Hình 12).



Hình 11. Demo F02: trước vá payload null byte đọc được `package.json.bak` và `coupons_2013.md.bak`; sau vá trả 403, tệp `.md` hợp lệ vẫn tải được



Hình 12. Kiểm thử bảo mật F01/F02 sau khi sửa: toàn bộ 11 ca đạt

Nguyên tắc/tiêu chuẩn đối chiếu: GDPR Điều 5(1)(f), 32(1)(b) (bảo mật, toàn vẹn khi xử lý). *Tệp bị lộ ở đây là cấu hình và mã giảm giá, không phải dữ liệu thẻ, nên không viện dẫn PCI-DSS.* **Pha DevSecOps:** Code → Test. **Kiểm soát (Security as Code):** phát hiện bằng SAST ở pha Code, chặn tái diễn bằng kiểm thử bảo mật tự động ở pha Test.

5.3. F03 — Lộ endpoint /metrics không xác thực

Khác F01/F02 (lỗi trong mã nguồn), F03 là **lỗi cấu hình triển khai**: mã ứng dụng giữ nguyên, rủi ro nằm ở file `docker-compose` để backend publish cổng ra host. Vì vậy, giống F04 dùng `docker_security.rego` cho Dockerfile, F03 **cần một chính sách Rego cho file compose** (`policy/compose.rego`) để phát hiện cấu hình sai ngay từ file, trước khi triển khai.

Trước khi sửa

Juice Shop mở sẵn `/metrics` (Prometheus) mà không yêu cầu đăng nhập, xuất ra các chỉ số như số người dùng, số đơn hàng và tổng số dư ví. Khi container publish thẳng cổng ứng dụng, **mọi chủ thể truy cập tới host** đều gọi được endpoint này và đọc số liệu vận hành — vi phạm nguyên tắc *giảm bề mặt tấn công và tối thiểu phơi bày*. Đây là một rủi ro thuộc khâu **triển khai/vận hành**. Cấu hình trước khi vá (`report/fixtures/compose.before-f03.yml`, rút gọn) đã có proxy nhưng backend vẫn publish cổng:

```
services:
  proxy:
    ports: ["127.0.0.1:3102:8080"]
    labels: {lab.exposure: edge}
  juice-shop:
    ports: ["127.0.0.1:3101:3000"]   # backend publish cổng host -> vòng qua proxy
```

Những cách kiểm tra sơ sài dễ bị qua mặt. Chỉ chặn `/metrics` ở proxy là chưa đủ: khi backend còn cổng host, người dùng gọi thẳng `127.0.0.1:3101/metrics` và vòng qua mọi luật đặt ở proxy. Kiểm tra kiểu "cổng chỉ bind loopback" cũng không bắt được lỗi này, vì `127.0.0.1:3101` vẫn là loopback. Còn gọi HTTP tới `/metrics` chỉ thấy lỗi **sau khi đã triển khai**, tức là dữ liệu đã lộ. Muốn chặn từ gốc thì phải có luật đọc chính file compose và chạy trước `docker compose up`.

Rủi ro nếu để nguyên, không vá. Mỗi hệ quả gắn với một yêu cầu an toàn cụ thể:

- **Lộ số liệu vận hành:** số người dùng, số đơn hàng, tổng số dư ví… mở cho mọi chủ thể truy cập được host, trái với yêu cầu *theo mặc định không mở dữ liệu cho số người không xác định* (GDPR Điều 25(2)).
- **Vô hiệu hóa kiểm soát ở proxy:** mọi route khác cũng tới được qua cổng backend, nên giới hạn tốc độ đăng nhập đặt ở proxy bị vòng qua và kẻ tấn công có thể dò mật khẩu không giới hạn.

Phát hiện. F03 được phát hiện ở hai lớp. Ở **mức cấu hình**, Conftest chạy `compose.rego` (luật `F03 NET-02`) trên file compose và chặn service backend có khóa `ports`; luật này là một cổng của pha Deploy trong pipeline, chạy trước `docker compose up`. Ở **mức vận hành**, demo `demos/tv3_f03_metrics.py` gọi `/metrics` từ host; trước khi sửa, endpoint trả về 200 kèm hàng trăm dòng số liệu (Hình 13).

Sau khi sửa

Việc xử lý gồm **hai phần tách bạch**: (1) **sửa chính cấu hình triển khai** (proxy và file compose), và (2) **viết chính sách kiểm tra tự động** (`compose.rego`) để chống tái diễn. Conftest đánh giá file compose theo luật Rego, trả PASS/FAIL kèm lý do và mã thoát tương ứng; việc sửa cấu hình được thực hiện riêng trong `docker-compose.yml` và `nginx.conf`.

(1) Sửa cấu hình triển khai. Bỏ cổng host của backend và chỉ để reverse proxy mở cổng loopback; tại proxy, `/metrics` bị chặn và tốc độ đăng nhập bị giới hạn:

```
location = /metrics { return 404; }
location = /rest/user/login { limit_req zone=login burst=5 nodelay; limit_req_status 429;
proxy_pass http://juice-shop:3000; }
```

```
proxy:
  ports: ["127.0.0.1:3102:8080"]
```

```
  labels: {lab.exposure: edge}       # dịch vụ biên duy nhất được publish cổng
juice-shop:
  labels: {lab.exposure: internal}   # BỎ khóa ports -> không truy cập trực tiếp từ host
  networks: [backend]
networks:
  backend: { internal: true }
```

Sau khi sửa, `/metrics` đi qua proxy trả về 404 và cổng backend cũ bị từ chối kết nối **từ host**. Đây là chặn ở lối vào từ bên ngoài chứ không phải khóa tuyệt đối: trong mạng internal, hệ thống giám sát vẫn đọc được metrics đúng mục đích (Hình 13).

(2) Viết chính sách thành mã (Policy as Code). Bộ luật Rego (`policy/compose.rego`) đọc file docker-compose (Conftest tự phân tích YAML) và duyệt từng service. Luật cốt lõi của F03 là **NET-02** theo kiểu *mặc định từ chối*: service nào có `ports` mà không tự khai nhãn `lab.exposure: edge` thì bị chặn, nên chỉ dịch vụ biên (reverse proxy) được nhận kết nối từ host. Luật đọc được cả ba dạng khai báo cổng của Compose (`"127.0.0.1:3102:8080"`, số `3000`, đối tượng `{target, published, host_ip}`), nên không lách được bằng cách đổi cú pháp. Đoạn luật chính:

```
edge(svc) if object.get(object.get(svc, "labels", {}), "lab.exposure", "") == "edge"

deny contains msg if {
  some name, svc in input.services
  count(ports(svc)) > 0     # service có publish cổng host
  not edge(svc)             # nhưng không phải dịch vụ biên
  msg := sprintf("[F03 NET-02] service '%s' publishes ports but is not an edge service ...",
[name])
}
```

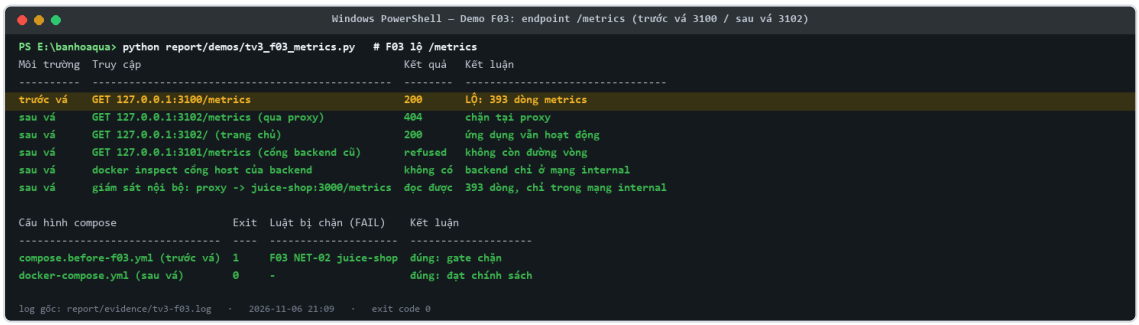
Bộ luật có 6 ca kiểm thử (`policy/compose_test.rego`), tất cả đạt, trong đó có ca "backend publish `127.0.0.1:3101` phải bị chặn dù đã bind loopback". Mã nguồn đầy đủ ở **Phụ lục B**.

Các luật và tiêu chuẩn đối chiếu:

Bảng 12. Luật kiểm tra cấu hình docker-compose — luật yêu cầu gì và dựa trên chuẩn nào

Luật	Luật yêu cầu gì (chặn khi vi phạm)	Tiêu chuẩn (ý nghĩa xem Phụ lục C)
F03 NET-02	Service **không** mang nhãn `lab.exposure: edge` thì **không** được publish cổng host, kể cả khi bind loopback (bắt buộc, `deny`)	GDPR Đ.25(2), 32(1)(b)
NET-01	Cổng đã publish **phải** bind `127.0.0.1`, không mở ra mọi giao diện mạng	GDPR Đ.32
CIS 5.4 / 5.31	**Không** chạy `privileged`; **không** mount `/var/run/docker.sock`	CIS Docker 5.4, 5.31
INLINE-SECRET	**Không** đặt mật khẩu/token trực tiếp trong `environment` (cho phép biến `*_FILE` trỏ tới secret)	PCI-DSS 8.2.1; GDPR Đ.32
Cảnh báo (`warn`)	Nên `cap_drop: [ALL]` và `read_only: true`; chỉ cảnh báo, không chặn	CIS Docker 5.3, 5.12

Trong pipeline, luật chạy ở cổng pha Deploy (`conftest test docker-compose.yml -p policy --namespace compose`); vi phạm thì exit 1 và pipeline **không chạy** `docker compose up`. Cùng ý NET-02 được kiểm lại ở pha Operate trên container đang chạy (`report/policy/runtime.rego`, đọc kết quả `docker inspect`), phòng trường hợp container được khởi chạy ngoài pipeline. Demo `tv3_f03_metrics.py` chạy Conftest trên hai cấu hình: `report/fixtures/compose.before-f03.yml` bị chặn đúng một luật `F03 NET-02` (exit 1), còn `docker-compose.yml` sau vá đạt (exit 0, chỉ một cảnh báo CIS 5.12) (Hình 13).



Hình 13. Demo F03: /metrics lộ khi truy cập trực tiếp, bị chặn sau khi sửa; Conftest chặn cấu hình compose trước vá

Nguyên tắc/tiêu chuẩn đối chiếu: GDPR Điều 25(2), 32(1)(b). **Pha DevSecOps:** Deploy → Operate. **Kiểm soát (Security as Code › Policy as Code):** luật NET-02 trong `compose.rego` chặn ở pha Deploy mọi cấu hình để backend mở cổng host; `runtime.rego` kiểm lại trên container đang chạy ở pha Operate.

5.4. F04 — Sai cấu hình an toàn của container

Khác ba lỗi trên, F04 **không phải lỗi của image Juice Shop gốc** (image distroless gốc đã chạy non-root và đạt bộ luật). Đây là một **fixture cấu hình sai chủ động dựng** (`report/fixtures/Dockerfile.vulnerable`) để kiểm tra *năng lực phát hiện* của bộ policy, sử dụng cả đầu vào "xấu" (chứng minh luật bắt được) và đầu vào "tốt" (chứng minh luật không báo nhầm).

Trước khi sửa

Một image đóng gói Juice Shop có thể vi phạm chính sách an toàn tối thiểu mà vẫn lọt qua nếu kiểm tra đặt sai chỗ. `Dockerfile.vulnerable` dưới đây minh họa bốn vi phạm như vậy:

```
FROM node:22-alpine
COPY . .
ENV API_TOKEN=supersecret123  # nhúng bí mật vào image (lộ qua docker history)
EXPOSE 3000 22 3306           # khai báo cổng 22 (SSH), 3306 (DB)
CMD ["npm", "start"]           # không USER (chạy root), không nhãn nguồn gốc
```

Những cách kiểm tra sơ sài dễ bị qua mặt. Kiểm tra kiểu "có lệnh `USER`" sẽ bỏ sót Dockerfile nhiều stage, vì `USER` ở stage builder không áp dụng cho image chạy thật. Kiểm tra `EXPOSE` chỉ đánh giá quy ước khai báo image, không phản ánh phơi nhiễm mạng thực tế (phải kiểm tại runtime). Nói chung, metadata tự khai trong image không thay cho bằng chứng thực thi.

Rủi ro nếu để nguyên, không vá. Mỗi vi phạm kéo theo một rủi ro an toàn cụ thể:

- **Chạy bằng root (LEAST-PRIVILEGE):** nếu về sau *giả định* tồn tại thêm một lỗ hổng **thực thi mã (RCE)** trong khi tiến trình chạy bằng root *bên trong* container, thiệt hại sẽ lớn hơn nhiều: kẻ tấn công dễ ghi đè file hệ thống, cài thêm công cụ, và nếu ghép thêm một lỗ hổng nhân hệ điều hành thì có thể **thoát container và leo thang đặc quyền** lên máy chủ. Lưu ý thực nghiệm F01 chỉ chứng minh **vượt xác thực**, chưa phải RCE; chạy non-root (UID 65532) là biện pháp *phòng thủ theo chiều sâu*, khoanh vùng thiệt hại cho những lỗ hổng giả định khác.
- **Khai báo cổng quản trị/CSDL (ATTACK-SURFACE):** `EXPOSE 22/3306` **tự nó không mở** cổng hay khởi động dịch vụ (EXPOSE chỉ mang tính tài liệu), nhưng là **tín hiệu** rằng image dự kiến chạy các dịch vụ này và thường đi kèm publish cổng khi vận hành. Luật chặn khai báo này để giữ *quy ước* image tối giản, hướng người viết Dockerfile không mang theo cổng nhạy cảm; còn phơi nhiễm **thực tế** phải kiểm bằng cấu hình runtime (cổng lắng nghe, publish, network policy), không suy ra từ EXPOSE.

- **Rủi ro chuỗi cung ứng (SUPPLY-CHAIN):** nếu thiếu nhãn nguồn gốc `org.opencontainers.image.source`/`revision`, khi có sự cố sẽ **khó truy vết image bắt nguồn từ commit nào** để điều tra hay thu hồi (đây là *metadata nguồn gốc tự khai*, hỗ trợ truy vết; **chưa đủ để kết luận đạt mức SLSA** — SLSA yêu cầu provenance được *sinh tự động*, mô tả cách build và được phân phối/kiểm chứng). Thêm vào đó, dùng `:latest` hoặc chưa ghim digest khiến image có thể **bị đổi âm thầm** giữa các lần build, làm bản chạy thật khác bản đã kiểm thử; còn lệnh `ADD` thì có thể kéo nội dung từ xa ngoài kiểm soát. Ghim digest cố định tham chiếu image, nhãn nguồn gốc hỗ trợ truy vết, COPY tránh kéo nội dung ngoài kiểm soát; đây là điều kiện cần cho build tái lập, chưa đủ nếu dependency và đầu vào build còn biến đổi.
- **Bí mật nhúng trong image (SECRET-IN-IMAGE):** đặt mật khẩu hay token trong `ENV`/`ARG` khiến bí mật **bị bake vào layer của ảnh**, và bất kỳ ai `docker pull` rồi chạy `docker history` đều đọc được, kể cả khi dòng đó đã bị xóa ở layer sau. Cách đúng là dùng *build secret* hoặc nạp biến môi trường lúc chạy. (Đây là **luật do nhóm bổ sung**.)

Sau khi sửa

Việc xử lý gồm **hai phần tách bạch**: (1) **sửa chính file triển khai** cho đúng cấu hình an toàn, và (2) **viết chính sách kiểm tra tự động** để chống tái diễn. Conftest đánh giá cấu hình theo luật Rego, trả PASS/FAIL kèm lý do và mã thoát tương ứng; việc sửa cấu hình được thực hiện riêng trong Dockerfile.

(1) Sửa file Dockerfile triển khai. So với bản lỗi, bản vá (`report/fixtures/Dockerfile.remediated`) khắc phục đúng từng vi phạm tương ứng:

```
FROM node:22-alpine
# THÊM nhãn nguồn gốc OCI  -> đạt SUPPLY-CHAIN (provenance)
LABEL org.opencontainers.image.source="https://github.com/juice-shop/juice-shop" \
      org.opencontainers.image.revision="${VCS_REF}"
RUN addgroup -S -g 10001 juicers && adduser -S -u 10001 -G juicers juicers
COPY --chown=10001:0 . .
USER 10001:10001          # THÊM USER non-root       -> đạt LEAST-PRIVILEGE
EXPOSE 3000               # BỎ cổng 22 và 3306        -> đạt ATTACK-SURFACE
CMD ["npm", "start"]
```

Cụ thể, bản vá thêm lệnh `USER` để chạy non-root, loại hai cổng quản trị/CSDL khỏi `EXPOSE`, bổ sung nhãn nguồn gốc OCI và **không đặt bí mật trong `ENV`/`ARG`** (dùng *build secret* hoặc nạp biến môi trường lúc chạy). Image triển khai thực tế của nhóm (`deploy/Dockerfile.hotfix`) lẫn Dockerfile gốc của Juice Shop đều đã thỏa những yêu cầu này.

(2) Viết chính sách thành mã (Policy as Code). Bộ luật Rego (`policy/docker_security.rego`) đặt mỗi luật đúng điểm thực thi để chặn các kiểu lách đã biết (vd `USER` ở stage builder không tính cho stage cuối; UID root kèm group như

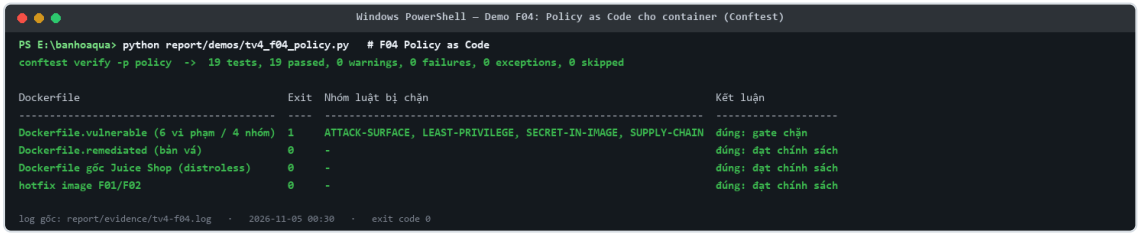
`0:1000`). Bộ luật có 13 ca kiểm thử, tất cả đạt (`conftest verify` 19/19). Mã nguồn đầy đủ ở **Phụ lục A**.

Các luật và tiêu chuẩn đối chiếu:

Bảng 13. Luật kiểm tra cấu hình container — luật yêu cầu gì và dựa trên chuẩn nào

Luật	Luật yêu cầu gì (chặn khi vi phạm)	Tiêu chuẩn (ý nghĩa xem Phụ lục C)
LEAST-PRIVILEGE	Stage cuối **phải** có `USER` không phải root (bắt buộc, `deny`; tách UID trước dấu `:`); khuyến nghị UID số (`warn` nếu không)	CIS Docker 4.1; PCI-DSS 2.2; NIST SP 800-190
ATTACK-SURFACE	**Không** được `EXPOSE` cổng quản trị/CSDL (22/23/3306/5432)	CIS Docker 5.8; NIST SP 800-190
SUPPLY-CHAIN	**Phải** có nhãn nguồn gốc OCI; **không** dùng `:latest`/`ADD`; nên ghim digest	OCI image-spec; CIS Docker Benchmark §4; NIST SSDF (SLSA: *tham chiếu*, chưa đánh giá đạt)
SECRET-IN-IMAGE *(bổ sung)*	**Không** đặt secret/credential trong `ENV`/`ARG`	CIS Docker 4.10; PCI-DSS 8.2.1; GDPR Đ.32

Lệnh `conftest verify` chạy toàn bộ chính sách đạt **19/19** ca kiểm thử (13 ca cho chính sách Dockerfile của F04 và 6 ca cho chính sách compose của F03). Kết quả cho thấy `Dockerfile.vulnerable` bị chặn bởi 4 nhóm luật (LEAST-PRIVILEGE, ATTACK-SURFACE, SUPPLY-CHAIN, SECRET-IN-IMAGE), bản vá thì đạt, và Dockerfile gốc distroless của Juice Shop cũng **đạt bộ luật** (Hình 14).



Hình 14. Demo F04: Dockerfile.vulnerable bị chặn bốn nhóm luật, bản vá và Dockerfile gốc đạt chính sách

Nguyên tắc/tiêu chuẩn đối chiếu: GDPR Điều 25(1), 32(1)(d); CIS Docker Benchmark 4.1. **Pha DevSecOps:** Build → Release. **Kiểm soát (Security as Code › Policy as Code):** bộ luật Rego chạy như một cổng kiểm tra trong pipeline.

6. Kết luận

Báo cáo minh họa quy trình **phát hiện – sửa – kiểm chứng** cho hai lỗi mã nguồn (F01 SQL injection ở đăng nhập, F02 Poison Null Byte làm lộ tệp sao lưu), một tình huống phơi nhiễm endpoint giám sát (F03) và một tình huống cấu hình container sai **chủ động dựng** để kiểm tra bộ policy (F04). Toàn bộ được thể hiện dưới dạng **Security as Code** — với **Policy as Code** (mở rộng thành **Compliance as Code**), SAST/SCA, kiểm thử bảo mật và giám sát là các kỹ thuật con — trong một chuỗi kiểm soát DevSecOps chạy được, kèm log gốc và mã bằng chứng (mục 4.2).

Chuỗi kiểm soát trải khắp vòng đời — phát hiện lỗ hổng SAST/SCA (Semgrep, Trivy) và kiểm thử động ở Code/Test, Policy as Code (Conftest) ở Build/Deploy, đến Compliance as Code (Dockle/CIS + `docker inspect`) và giám sát Prometheus/Grafana ở Operate. Phần đóng góp của nhóm — trong phạm vi bốn tình huống — gồm: bản vá F01/F02 kèm kiểm thử hồi quy; reverse proxy và giới hạn tốc độ cho F03; bộ luật Rego (Dockerfile + compose + runtime, 26 unit test) cho F04 và F03; quét **CVE/SCA (Trivy)** và kiểm **CIS image (Dockle)** ở pha Release; và **giám sát vận hành bằng Prometheus/Grafana kèm cảnh báo** fire thật ở pha Operate. Kho mã Juice Shop gốc **đã có** CI/CD, CodeQL và tạo SBOM; báo cáo không khẳng định dự án gốc thiếu tự động hóa bảo mật, đóng góp ở đây là các kiểm soát **bổ sung, hướng dữ liệu cá nhân**.

Toàn bộ các cổng trên được gói thành **một pipeline 7 pha gắn với Git** (mục 4.4): hook `pre-commit`/`pre-push` chặn ngay trên máy thành viên, GitHub Actions chạy lại đủ 7 pha làm cổng cuối; cả 7 tình huống đưa lỗi F01–F04 trở lại đã thử đều bị chặn đúng pha.

Các giới hạn còn lại và việc cần làm tiếp được tổng hợp trong tài liệu đi kèm `report/TONG_KET_PHAN_CONG.md` (mục 2.4): chưa bật branch protection cho `main`, stack giám sát chưa đạt chính `compose.rego`, Grafana ẩn danh quyền Admin, và cổng Test vẫn qua nếu không còn ca kiểm thử nào.

7. Tài liệu tham khảo

- OWASP Juice Shop v20.2.0 — https://owasp.org/www-project-juice-shop/ (truy cập 10/2026).
- GDPR, Regulation (EU) 2016/679, Điều 5, 25, 32.
- PCI-DSS v3.2.1 (đối chiếu lịch sử — đã nghỉ 31/3/2024; bản hiện hành v4.0.1), Yêu cầu 2.2, 3.4, 8.2.1 — pcisecuritystandards.org.
- CIS Docker Benchmark (§4 Container Images, §5 Runtime); OWASP Top 10 2021; NIST SP 800-190; NIST SSDF (SP 800-218); OCI image-spec; SLSA v1.x (slsa.dev).

- Công cụ (phiên bản dùng trong thực nghiệm): Semgrep 1.180.0, Gitleaks v8.30.1, Conftest/OPA v0.71.1, node:test + Supertest, Trivy 0.76.1, Dockle 0.4.15, Prometheus 3.15.1, Grafana 13.3.0.

Phụ lục A — Mã nguồn chính sách Dockerfile (docker_security.rego)

```
# Policy as Code cho Dockerfile của Juice Shop.
# Mỗi luật đặt tại điểm thực thi đúng thay vì các "original check" dễ bị qua mặt:
#   Least privilege  -> USER ở STAGE CUỐI, UID số, không phải root (thay vì "có USER bất kỳ trong
file")
#   Firewall/RBAC    -> cổng quản trị/DB không được EXPOSE (giảm bề mặt tấn công)
#   Supply chain     -> metadata nguồn gốc (nhãn OCI source+revision; hỗ trợ truy vết, CHƯA đánh
giá đạt SLSA) + không :latest, không ADD, cảnh báo base chưa ghim digest
#   Secret hygiene   -> KHÔNG nhúng bí mật (ENV/ARG) vào image (luật riêng của nhóm)
# Mỗi thông báo vi phạm trích sẵn điều khoản chuẩn quốc tế áp dụng (CIS Docker, PCI-DSS, NIST,
OCI/SLSA).
#   chạy: conftest test <Dockerfile> -p policy/ --namespace dockerfile
package dockerfile

import rego.v1

# Danh sách cổng quản trị/CSDL (SSH, telnet, Docker daemon, MySQL, Postgres, Redis, Mongo).
forbidden_ports := {"22", "23", "2375", "2376", "3306", "5432", "6379", "27017"}

required_labels := {"org.opencontainers.image.source", "org.opencontainers.image.revision"}

# UID là phần trước dấu ':' trong 'USER uid[:gid]'. Nhận diện root ('0' hoặc 'root') độc lập với
group
# kèm theo, nên 'USER 0:1000' cũng bị nhận diện là root.
uid_of(u) := split(u, ":")[0]

# Chỉ STAGE CUỐI tạo ra image chạy thật; USER ở stage builder không áp dụng cho runtime.
final_stage := max([ins.Stage | some ins in input])

final_users := [ins.Value[0] | some ins in input; ins.Stage == final_stage; ins.Cmd == "user"]

last_user := final_users[count(final_users) - 1] if count(final_users) > 0

# LEAST-PRIVILEGE ----------------------------------------------------------
deny contains msg if {
        count(final_users) == 0
        msg := "LEAST-PRIVILEGE: final stage has no explicit USER; policy requires an explicit
non-root USER (otherwise the container may run as root). (CIS Docker 4.1; PCI-DSS 2.2; NIST SP
800-190)"
}

deny contains msg if {
        uid_of(last_user) in {"root", "0"}
        msg := sprintf("LEAST-PRIVILEGE: final USER '%s' runs as root (UID 0), regardless of
group. (CIS Docker 4.1; PCI-DSS 2.2)", [last_user])
}

warn contains msg if {
        last_user
        not uid_of(last_user) in {"root", "0"}
        not regex.match(`^[0-9]+(:[0-9]+)?$`, last_user)
        msg := sprintf("LEAST-PRIVILEGE: final USER '%s' is not a fixed numeric UID. (CIS Docker
4.1)", [last_user])
}

# ATTACK-SURFACE (firewall / RBAC) -----------------------------------------
port_number(raw) := split(raw, "/")[0] # bỏ hậu tố /tcp, /udp
```

```
deny contains msg if {
        some ins in input
        ins.Cmd == "expose"
        some raw in ins.Value
        port_number(raw) in forbidden_ports
        msg := sprintf("ATTACK-SURFACE: EXPOSE '%s' covers administrative/database port %s. (CIS
Docker 5.8; NIST SP 800-190)", [raw, port_number(raw)])
}

# SUPPLY-CHAIN — provenance khai báo (traceability; không thay cho chữ ký/attestation) --
label_blob := concat(" ", [concat(" ", ins.Value) | some ins in input; ins.Cmd == "label"])

deny contains msg if {
        some req in required_labels
        not contains(label_blob, req)
        msg := sprintf("SUPPLY-CHAIN: missing provenance label '%s' (metadata nguồn gốc hỗ trợ
truy vết; chưa đánh giá đạt mức SLSA). (OCI image-spec; NIST SSDF)", [req])
}

# SUPPLY-CHAIN — base image (không :latest/ADD, ghim digest) -----------------
deny contains msg if {
        some ins in input
        ins.Cmd == "from"
        endswith(ins.Value[0], ":latest")
        msg := sprintf("SUPPLY-CHAIN: base image '%s' uses :latest; pin a version or digest. (CIS
Docker Benchmark §4 Container Images; NIST SSDF)", [ins.Value[0]])
}

deny contains msg if {
        some ins in input
        ins.Cmd == "add"
        msg := "SUPPLY-CHAIN: use COPY instead of ADD to avoid remote/implicit extraction. (CIS
Docker 4.9)"
}

warn contains msg if {
        some ins in input
        ins.Cmd == "from"
        not contains(ins.Value[0], "@sha256:")
        not endswith(ins.Value[0], ":latest")
        msg := sprintf("SUPPLY-CHAIN: base image '%s' is not pinned by digest (digest pin khác
với Content Trust/CIS 4.5). (CIS Docker Benchmark §4; NIST SSDF)", [ins.Value[0]])
}


secret_key := `(?i)(password|passwd|secret|token|api[_-]?key|access[_-]?key|private[_-]?
key|credential)`

deny contains msg if {
        some ins in input
        ins.Cmd == "env"
        some i, key in ins.Value
        i % 3 == 0
        i + 1 < count(ins.Value)
        regex.match(secret_key, key)
        ins.Value[i + 1] != ""
        msg := sprintf("SECRET-IN-IMAGE: ENV '%s' hardcodes a credential into the image (exposed
via docker history); use build secrets/runtime env. (CIS Docker 4.10; GDPR Art.32; PCI-DSS
8.2.1)", [key])
}

deny contains msg if {
        some ins in input
        ins.Cmd == "arg"
        some tok in ins.Value
```

```
        contains(tok, "=")
        regex.match(secret_key, split(tok, "=")[0])
        split(tok, "=")[1] != ""
        msg := sprintf("SECRET-IN-IMAGE: ARG '%s' bakes a credential default into the image. (CIS
Docker 4.10; GDPR Art.32; PCI-DSS 8.2.1)", [split(tok, "=")[0]])
}
```

Phụ lục B — Mã nguồn chính sách docker-compose (compose.rego)

```
package compose

import rego.v1

# Input: file docker-compose (conftest tự đọc YAML).
# ports chấp nhận cả 3 dạng của Compose: "127.0.0.1:3102:8080", 3000, {target, published,
host_ip}.
# Thông báo vi phạm trích điều khoản chuẩn quốc tế (GDPR, PCI-DSS, CIS Docker).

host_ip(p) := split(p, ":")[0] if {
        is_string(p)
        count(split(p, ":")) == 3
}

host_ip(p) := "" if {
        is_string(p)
        count(split(p, ":")) < 3
}

host_ip(p) := "" if is_number(p)

host_ip(p) := object.get(p, "host_ip", "") if is_object(p)

ports(svc) := object.get(svc, "ports", [])

# Dịch vụ được thiết kế để nhận kết nối từ host (reverse proxy, dashboard) phải tự khai báo nhãn.
edge(svc) if object.get(object.get(svc, "labels", {}), "lab.exposure", "") == "edge"

# F03 / NET-02: backend không được publish cổng. Nếu backend có cổng host, mọi route (kể cả
/metrics)
# có thể bị truy cập vòng qua proxy, làm vô hiệu luật chặn /metrics và giới hạn tốc độ ở proxy.
deny contains msg if {
        some name, svc in input.services
        count(ports(svc)) > 0
        not edge(svc)
        msg := sprintf("[F03 NET-02] service '%s' publishes ports but is not an edge service
(label lab.exposure=edge); backend must only be reachable through the proxy", [name])
}

# NET-01 (GDPR Art.32): cổng publish chỉ được bind loopback trong lab.
deny contains msg if {
        some name, svc in input.services
        some p in ports(svc)
        host_ip(p) != "127.0.0.1"
        msg := sprintf("[GDPR Art.32 NET-01] service '%s' publishes %v on all interfaces", [name,
p])
}

deny contains msg if {
        some name, svc in input.services
        svc.privileged == true
        msg := sprintf("[CIS Docker 5.4] service '%s' runs privileged", [name])
}

deny contains msg if {
        some name, svc in input.services
        some v in object.get(svc, "volumes", [])
```

```
        contains(sprintf("%v", [v]), "docker.sock")
        msg := sprintf("[CIS Docker 5.31] service '%s' mounts the Docker socket", [name])
}

# Biến môi trường có thể là danh sách "KEY=value" hoặc map {KEY: value}; *_FILE trỏ tới secret
thì được phép.
env(svc) := [e | some e in svc.environment] if is_array(svc.environment)

env(svc) := [sprintf("%s=%v", [k, v]) | some k, v in svc.environment] if
is_object(svc.environment)

env(svc) := [] if not svc.environment

deny contains msg if {
        some name, svc in input.services
        some e in env(svc)
        regex.match(`(?i)^[A-Z0-9_]*(PASSWORD|SECRET|TOKEN|API_KEY)=.+`, e)
        msg := sprintf("[PCI-DSS 8.2.1 / GDPR Art.32] service '%s' has an inline secret in
environment: %s", [name, split(e, "=")[0]])
}

drops_all_caps(svc) if "ALL" in object.get(svc, "cap_drop", [])

warn contains msg if {
        some name, svc in input.services
        not drops_all_caps(svc)
        msg := sprintf("[CIS Docker 5.3] service '%s' does not drop capabilities", [name])
}

warn contains msg if {
        some name, svc in input.services
        not svc.read_only
        msg := sprintf("[CIS Docker 5.12] service '%s' has a writable root filesystem", [name])
}
```

Phụ lục C — Giải thích tiêu chuẩn và điều khoản tham chiếu

Bảng tra ý nghĩa (**yêu cầu gì**) của mọi tiêu chuẩn/điều khoản được trích trong báo cáo. Phần thân chỉ ghi mã; ý nghĩa tra tại đây.

Bảng 14. Tra cứu — mỗi tiêu chuẩn/điều khoản tham chiếu yêu cầu gì

Điều khoản	Yêu cầu / ý nghĩa tóm gọn
GDPR Điều 5(1)(f)	Nguyên tắc *toàn vẹn & bảo mật*: xử lý dữ liệu một cách an toàn
GDPR Điều 25	Bảo vệ dữ liệu *từ thiết kế* (khoản 1) và *theo mặc định* (khoản 2)
GDPR Điều 32	Biện pháp kỹ thuật & tổ chức bảo đảm an toàn khi xử lý — (1)(b) bí mật/toàn vẹn, (1)(d) kiểm thử & đánh giá định kỳ
OWASP A03:2021 / CWE-89	*Injection* — tiêm nhiễm dữ liệu vào câu lệnh (SQL injection)
OWASP A01:2021 / CWE-158	*Broken Access Control* — xử lý sai null byte (`%00`) làm vượt kiểm tra loại tệp (F02)
CWE-180	Kiểm tra đầu vào *trước khi* chuẩn hóa — tên được kiểm tra khác tên thực sự dùng
PCI-DSS 2.2	Cấu hình tham số an toàn hệ thống, chạy với đặc quyền tối thiểu
PCI-DSS 3.4	Làm số thẻ (PAN) không đọc được ở mọi nơi lưu trữ
PCI-DSS 8.2.1	Không lưu thông tin xác thực (credential) ở dạng đọc được
CIS Docker 4.1	Chạy container bằng user riêng, **không phải root**
CIS Docker 4.9	Dùng `COPY` thay cho `ADD` (tránh kéo nội dung từ xa ngoài kiểm soát)
CIS Docker 4.10	Không lưu bí mật trong Dockerfile
CIS Docker 5.3	Loại bỏ Linux capabilities không cần (`cap_drop: ALL`)
CIS Docker 5.4	Không chạy container ở chế độ `privileged`
CIS Docker 5.8	Chỉ mở đúng cổng cần thiết (không mở cổng quản trị/CSDL)
CIS Docker 5.12	Gắn hệ thống tệp gốc ở chế độ chỉ đọc (`read_only`)
CIS Docker 5.31	Không mount Docker socket (`/var/run/docker.sock`) vào container
OCI image-spec	Chuẩn định dạng ảnh container + nhãn metadata nguồn gốc
SLSA	Khung toàn vẹn chuỗi cung ứng. Mức Build L1 yêu cầu provenance được *sinh tự động* và phân phối; nhãn OCI tự khai trong báo cáo **chưa đạt** L1, chỉ dùng để tham chiếu
NIST SP 800-190	Hướng dẫn an toàn cho container